

where a command byte and a few address bytes are followed by a whole 'page' worth of data bytes on the same 8-bit data interface. That is, the bytes are serially transmitted between the flash device and the microprocessor," shares Rosenboim.

"The more recent NOR flash based devices—those with multi-level cells—have become much slower to erase than their predecessors. Four seconds to erase a 128k block is truly slow by any measure, but it appears these are made necessary by the changes in technology that lead to lower cost per bit and higher densities," he adds.

On the positive side, these newer NOR parts have an improved erase-suspend ability. So with a bit more complexity of the software layer, one can mitigate this problem by suspending the erase process to let other activities execute while performing the erase operations in the background effectively. This is much less of a problem than all operations being blocked for the duration of a 'block erase' operation.

If the application requires a lot of inputs/outputs (I/Os) for booting, one may opt for industrial-grade flash devices that can handle the demanding speed.

Endurance: How many times can you rewrite/erase?

The critical use of flash memories in consumer products like MP3 players, USB pen drives and digital storage TV sets is to rewrite the information onto the memory. What comes into consideration here is the limitation of finite program/erase cycles.

Like any other erasable memory, flash devices have a limited number of erase/write cycles that they can withstand without failure. The reason for the limitation depends on either the charge-trapping characteristics or the dielectric-breakdown characteristics of the tunnel oxide. This introduces a term called 'endurance.'

Brian Beckham, manager-software



Intel's first 34nm NAND flash solidstate drive

design, Lester Electrical, says, "While using flash devices, we need to ensure that we stay below 50k writes within the expected product life." Though many devices claim a higher number of cycles, typically 100k, it is better to design within the life expectancy of the flash device than to try and extend it, he feels.

There is no guarantee to how long the parts or blocks will work. As long as the memory is used to store the program code, it is very unlikely that the application will reach the maximum guaranteed erase cycles. If you really think of it—how many times do you upload a new design code for the application? But when using the Flash for data storage, the entire game changes. So the designers must carefully calculate the worst-case scenario.

Let us take the example of a consumer electronics device that stores configuration settings with each power-down cycle. Only the change in the setting is stored to optimise the code.

If we assume that it uses two 16k flash sectors and the configuration data occupies 32 bytes, a total of 10,000 entries can be made before both the sectors go through an erase cycle. So a hundred million writes can be made before the system reaches the maximum number of erase cycles.

You can do the basic maths to calculate the maximum number of writes in a day for the device to last up to ten years.

"You can also implement a battery back-up cache RAM section to decrease the write cycles to the flash memory,"

suggests David Quinn, a hardware design and firmware engineer, QED.

Design considerations

Flash memories are making inroads into almost every consumer application either for code storage or for data storage. Depending on the type of the application, there are several aspects that need to be considered while using Flash in an embedded application design.

Understanding the write/erase cycles will help you design the product for life expectancy. But sometimes harsh environments may limit the lifetime of the products by affecting their components.

The embedded designer should be aware of the page size as well as erase/write time. Pothoof says, "The page size, along with data layout, is important because you don't want to erase a 1k page to update a single byte." The designer should also pay attention to the kind of power plan present while doing the updates. "Having a user pull the battery when you are in the middle of an erase cycle can lead to some very interesting and hard to find errors," explains Pothoof.

Temperature is another important consideration. The flash drive—the most stable device in optimal conditions—shows erratic behaviour at extremely high or low temperatures. Issues like corrupt data, slow response time and system error can occur.

"We also use write verification code with checksums, trap all errors from the device and apply other meth-